Task for the Master's Thesis

Danje Petersen

November 3, 2025

1 Motivation

Modern computers come with a set of processors that can run simultaneously. To utilize this for their advantage, developers started to design their programs to use multiple concurrently running threads. Since these threads might still operate on the same memory it is possible to encounter problems like data-races when executing these programs. Data races occur when two or more threads access the same memory space and at least one of them uses a write operation. The classic example is the following:

```
Init:x=y=0;

x=1; || y=1;
a=y; || b=x;
```

Figure 1: Example of two threads using the same variables. Depending on how the threads are scheduled, the resulting values could feature any combination of 1 and 0 for the variables a and b.

Since this might lead to inconsistent results and therefore unpredictable behaviour, programming languages and cpu architectures provide memory consistency models. If the developer confines themselves to these rules, they get certain guarantees in return. These memory consistency models describe how threads are allowed to interact with each other when it comes to shared memory.

TODO ▶*Warum ist es wichtig die Modelle zu verifizieren?*◀

2 Current Research

TODO ▶*Einordnung in ein weiter gefasstes Feld von Memory Consistency*◀

TODO ▶*Fragen die sich in diesem Forschungsfeld stellen (siehe Kater paper)*◀

TODO ▶*was bedeutet "weak" memory?*◀

In 2014 Alglave et al. proposed an axiomatic framework for modelling weak memory and the corresponding language CAT [1]. Axiomatic models work by constraining relations on memory access.

$$
\begin{aligned}
\langle cat \rangle &:= \langle constraint \rangle \mid \langle cat \rangle \ \langle cat \rangle \\
\langle constraint \rangle &:= \texttt{acyclic} \ \langle rel \rangle \mid \texttt{irreflexive} \ \langle rel \rangle \\
\langle rel \rangle &:= \langle base_rel \rangle \mid \langle rel \rangle.\langle rel \rangle \mid \langle rel \rangle^* \mid \langle rel \rangle \cup \langle rel \rangle \mid \langle rel \rangle \cap \langle rel \rangle \\
&\quad \mid \langle rel \rangle^{-1} \mid \langle rel \rangle^+ \mid \langle rel \rangle? \mid \langle base_set \rangle \times \langle base_set \rangle \mid \langle rel \rangle \setminus \langle rel \rangle \mid \sim \langle rel \rangle \\
\langle base_rel \rangle &:= \text{po} \mid \text{rf} \mid \text{rmw} \mid \text{loc} \mid \text{ext} \mid \text{id} \mid \emptyset \\
\langle base_set \rangle &:= \text{R} \mid \text{W} \mid \text{F} \mid \text{B} \mid \text{M} \mid \text{IW} \mid \text{E}
\end{aligned}
$$

Figure 2: The CAT language when including the red coloured terms, when excluding them it is the fragment called rational CAT. $\langle rel \rangle^+$ is the transitive closure, $\langle rel \rangle?$ the reflexive closure, $\langle rel \rangle^*$ the reflexive and transitive closure, $\sim \langle rel \rangle$ the complement [1].

Some very basic examples for these relations are the *program order* (po) which represents the sequence of events of a single thread and *read-from* (rf), displaying for every read which write determined the value it sees. Upon these and some more base relations, memory consistency models can be expressed with CAT as can be seen in fig. 2.

While the tool that Alglave et al. developed allows to test certain programs in a given memory model, verifying these models is a different issue. Verification of the memory consistency model means checking if a given model guarantees certain properties for every program that can be executed under it. The first tool to tackle this issue was KATER [3] which covered only a fragment of CAT that could be transformed into Kleene algebra with tests. This fragment excluded set difference, complement, intersections, converses and the construction of relations from sets.
The next year Koutsouridis et al. presented KATI [4] which is able to handle converses and some intersections.

The first tool that addressed all intersections as well as inverses and construction of relations from sets (called rational CAT, see fig. 2) was developed by the supervisors of this master's thesis in 2025 [2]. In their yet to be published work, they presented a logical theory of relations where the unsatisfiability of a formula is equivalent to the model fulfilling a condition. The formula consists of a negated subformula representing the memory model and a subformula for the condition.
The formula is then decomposed until there exists a program execution that fulfills it or it is determined unsatisfiable.

3 Task

The decomposition used in [2] might result in infinite paths when unrolling Kleene stars. This leads to the authors using a cyclic proof system that returns to a node in the decomposition tree when it is equal to the current node. The task for this master's thesis is to replace this cyclic proof system with one that uses invariants and is neither cyclic nor infinite. Additionally the new proof system is supposed to directly handle inclusion between memory models and does not need to resolve them to the event level. The student is supposed to create a proof system that works reasonably well and ideally proof it's completeness and soundness.

References

[1] Jade Alglave, Luc Maranget, and Michael Tautschnig. "Herding Cats: Modelling, simulation, testing and data mining for weak memory". In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014), pp. 1–74. ISSN: 0164-0925. DOI: 10.1145/2627752.

[2] Jan Grünke and Roland Meyer. "RAT-CAT-SAT: Model Checking Memory Consistency Models". In: (2025).

[3] Michalis Kokologiannakis, Ori Lahav, and Viktor Vafeiadis. "Kater: Automating Weak Memory Model Metatheory and Consistency Checking". In: *Proceedings of the ACM on Programming Languages* 7.POPL (2023), pp. 544–572. DOI: 10.1145/3571212.

[4] Aristotelis Koutsouridis, Michalis Kokologiannakis, and Viktor Vafeiadis. "Automating Memory Model Metatheory with Intersections: Schloss Dagstuhl – Leibniz-Zentrum für Informatik". In: *LIPIcs, Volume 311, CONCUR 2024* 311 (2024). DOI: 10.4230/LIPICS.CONCUR.2024.33.